How Matrix Size Affects the Runtime and Memory of Naive Matrix Multiplication in Python, Java and C

Author: Joel López Guillén
Assignment: 1 — Basic Matrix Multiplication in Different Languages
Course: BIG DATA, Grado en Ciencia e Ingeniería de Datos, Universidad de Las Palmas de Gran Canaria, 2026-2027
Repository: https://github.com/JoelLopezGuillen/big-data-matrix-multiplication

---

Abstract

This study measures how the matrix size *n* affects the runtime and peak memory of the same naive i-j-k triple-loop dense matrix multiplication implemented in C, Java and Python on a single laptop (Intel Core i7-12650H, Windows 11). All implementations read identical float64 input matrices, were validated against NumPy, and were timed over five repetitions after warm-up runs, reporting the median and interquartile range. Runtime did not follow a single cubic curve: the time per inner-loop iteration stayed roughly constant while matrix B, which is read column by column, fitted in one cache level and rose sharply each time it exceeded one (in C: 0.27, 0.5, 2.0 and 13.3 ns per iteration). C and Java performed similarly from n = 256 to 512, while CPython was 36–112 times slower than C at the sizes where both completed. The largest feasible sizes within a 900 s per-process budget were 2048 for C and 1024 for Java and Python; a supplementary run showed that the Java timeout at n = 2048 was caused by the budget design, not by infeasibility. These conclusions apply only to the tested implementations, settings, inputs and machine.

1. Introduction and objective

Dense matrix multiplication is a standard benchmark kernel because its arithmetic work grows as $O(n^3)$ while its data grows as $O(n^2)$, so it stresses both the processor and the memory hierarchy [3], [11]. Implementing the same algorithm in languages with different execution models (ahead-of-time compiled C, JIT-compiled Java and interpreted CPython) makes it possible to observe how execution model and memory layout affect performance under controlled conditions.

Research question. How does the matrix size *n* affect the runtime and peak memory of the same naive triple-loop dense matrix multiplication implemented in Python, Java and C, on one fixed machine?

Hypotheses (stated before the main results were analysed).

- **H1.** Runtime grows approximately as n^3, i.e. by a factor of about 8 each time n doubles.
- **H2.** C and Java, which execute native machine code, are much faster than CPython, which interprets bytecode.
- **H3.** In Python, interpreter overhead dominates each operation, so memory-hierarchy effects should be smaller than in C and Java and growth should stay closer to ×8.

2. Background

Naive matrix multiplication. To multiply two n × n matrices A and B, each element C[i][j] of the result is obtained by multiplying row i of A by column j of B, element by element, and adding the products. Using three nested loops (i, j and k), this requires n^3 multiplications and additions. Therefore, when n doubles, the amount of work grows by $2^3 = 8$.

How matrices are stored. In memory, a matrix is stored row by row: all the elements of row 0, then those of row 1, and so on. This means that reading a row of A is fast, because its elements are next to each other. Reading a column of B is slower, because each element is n positions away from the previous one.

Memory hierarchy. The processor does not read data directly from main memory every time, because that would be too slow. It keeps recently used data in small, fast memories called caches (L1, L2 and L3), each one bigger but slower than the previous one [3], [4]. Data is moved between these levels in blocks of 64 bytes called cache lines. When reading a row, one cache line brings eight consecutive values; when reading a column of B, almost every access needs a different cache line. While the data fits in a cache, access is fast. When it no longer fits, the processor has to fetch it from the next, slower level. On the test machine, each performance core has a 48 KiB L1 data cache and a 1.25 MiB L2 cache, and all cores share a 24 MiB L3 cache [2], [12].

How each language runs the code. C is translated into machine code before it runs. Java is compiled to bytecode, which the JVM first interprets and then, while running, its JIT compiler turns the most frequently used code into optimised machine code [5]. This is why Java needs some warm-up runs before measuring [10]. Python (CPython) also compiles the code to bytecode, but executes it one instruction at a time with an interpreter and treats every number as an object, which adds extra work to each operation.

Measuring correctly. A reliable benchmark must use a monotonic timer (one that never goes backwards), discard the warm-up runs, and use the result of the computation; otherwise the compiler could skip the computation entirely because its result is never used [1], [10].

3. Methodology

3.1 Implementations

All three implementations use the same algorithm: the i-j-k triple loop, accumulating the dot product in a local variable and then writing `C[i][j] = sum`. No optimised matrix multiplication library is used in any implementation.

- **C and Java** store each matrix as a flat row-major `double` array of length n^2, indexed as `i*n + k`.
- **Python** uses a list of lists of `float`, the idiomatic representation in pure Python. This is an unavoidable language-specific difference: each element is an 8-byte pointer to a separate float object (about 24 bytes), instead of an 8-byte value stored contiguously.
- The Python version stores the current row of A in a local variable (`a_row = a[i]`) outside the inner loops. gcc performs an equivalent optimisation automatically at `-O2`.
- The output matrix is allocated and zero-initialised inside the multiplication function (`calloc` in C, `new double[]` in Java, a list comprehension in Python).

3.2 Inputs

To compare the three languages fairly, all of them must multiply exactly the same matrices. Each programming language has its own random number generator, so generating the matrices separately in C, Java and Python would produce different inputs. Instead, the input matrices A and B were generated once for each size by a Python script (`data/generate.py`) and saved to files that all three implementations read [1].

The values are random numbers between -1 and 1 (uniform distribution over [-1, 1)), stored as 64-bit floating-point numbers (float64). They were generated with NumPy's `default_rng` generator using fixed seeds (42 for A and 43 for B), so running the script again produces exactly the same matrices. The files store the raw binary values row by row (little-endian, row-major order), which avoids any loss of precision from converting numbers to text.

3.3 Environment

Item	Value
CPU	Intel Core i7-12650H: 6 performance cores (P-cores, Hyper-Threading) and 4 efficiency cores (E-cores); 10 physical cores, 16 logical processors
Caches (P-core)	L1d 48 KiB and L2 1.25 MiB per core; L3 24 MiB shared; 64-byte lines (reported by Windows [12])
RAM	15.7 GiB
OS	Windows 11 (build 10.0.26200)
C	gcc 16.1.0 (MSYS2 UCRT64), flags `-O2 -std=c11 -Wall -Wextra`
Java	Oracle JDK 21.0.10 (HotSpot), compiled with `javac --release 21`, run with `-Xms4g -Xmx4g -XX:+UseSerialGC`
Python	CPython 3.13.7
Measurement tools	psutil 7.2.2, NumPy 2.5.4 (reference only)

Because the i7-12650H is a hybrid processor, it has two kinds of cores: fast performance cores (P-cores) and slower efficiency cores (E-cores). The operating system can move a running program from one core to another at any time. If this happened during a measurement, part of the run would be timed on a fast core and part on a slow one, making the results unreliable. To avoid this, we first checked which logical processors belong to the P-cores using a Windows function (`GetSystemCpuSetInformation`) [9]: processors 0–11 are P-cores and 12–15 are E-cores. Every benchmark process was then restricted to processors 0–11 and given high priority, so that other programs would interfere as little as possible. All implementations use a single thread. During the runs, the laptop was plugged in and heavy applications were closed.

These settings were the same for every configuration of the main run.

3.4 Measurement protocol

- **Timing boundary.** Only the multiplication kernel is timed, including allocation and zero-initialisation of the output matrix. Loading the input files happens before the timed region.
- **Timers.** Monotonic high-resolution timers were used: `QueryPerformanceCounter` in C (10 MHz, i.e. 100 ns resolution on this machine) [8], `System.nanoTime` in Java, and `time.perf_counter_ns` in Python [6].

- **Process structure.** Each (language, n) configuration ran as one process that loaded the inputs, performed the warm-up runs and then five measured repetitions. Warm-up runs were excluded from the samples: 5 for Java (to let the JIT compiler optimise the kernel) and 1 for C and Python (to warm caches and memory pages).
- **Result consumption.** After each multiplication, the sum of all elements of C (checksum) was computed outside the timed region and reported, so the computation cannot be eliminated.
- **Memory metric.** Peak working set (`peak_wset`, the peak amount of physical memory used by the process) was read with psutil [7] by an external harness for all three languages. Because Windows only exposes this value while the process exists, each benchmark process prints its results and then waits on standard input; the harness reads the peak working set and then releases the process. This metric covers the whole process: runtime (JVM or interpreter), loaded inputs and output.
- **Statistics.** Every raw measurement was stored in CSV. For each configuration the median and the interquartile range (IQR = Q3 − Q1) of the five repetitions are reported.

3.5 Correctness

Before benchmarking, we checked that every implementation computes the correct result. Each one passed unit tests with small cases of known answer: multiplication by the identity matrix, by a zero matrix, and two hand-calculated examples (2 × 2 and 3 × 3).

We also compared the results of all three implementations for n = 64 and n = 128 with NumPy, using the script `cross_check.py`. Exact equality is not expected, because computers round decimal numbers and NumPy adds the values in a different order than our loop. We therefore accepted tiny differences (`numpy.allclose` with a relative tolerance of 10^{-9} and an absolute tolerance of 10^{-12}), strict enough to detect any real error. The largest difference reported by `cross_check.py` was 5.3×10^{-15}, far below the tolerance.

3.6 Sizes and budget

The declared size sequence was n = 64, 128, 256, 512, 1024, 2048 and 4096. Each process had a budget of 900 s, covering warm-ups and the five repetitions. A configuration that exceeded the budget was killed and recorded as a timeout with its limit, and larger sizes for that language were skipped, since doubling n multiplies the work by eight. Memory was not a limiting factor: three float64 matrices at n = 4096 need about 384 MiB.

4. Results

4.1 Runtime

Table 1. Kernel runtime: median and IQR of 5 repetitions (seconds). Timeouts at 900 s per process.

n	C median	C IQR	Java median	Java IQR	Python median	Python IQR
64	0.0000696	0.0000012	0.000448	0.000029	0.00781	0.00002
128	0.00111	0.00002	0.00151	0.00001	0.0613	0.0002
256	0.00837	0.00004	0.00941	0.00016	0.546	0.004
512	0.270	0.003	0.277	0.011	9.60	0.94
1024	2.18	0.04	3.78	0.23	113.6	11.3
2048	114.5	5.5	timeout	—	timeout	—
4096	timeout	—	skipped	—	skipped	—

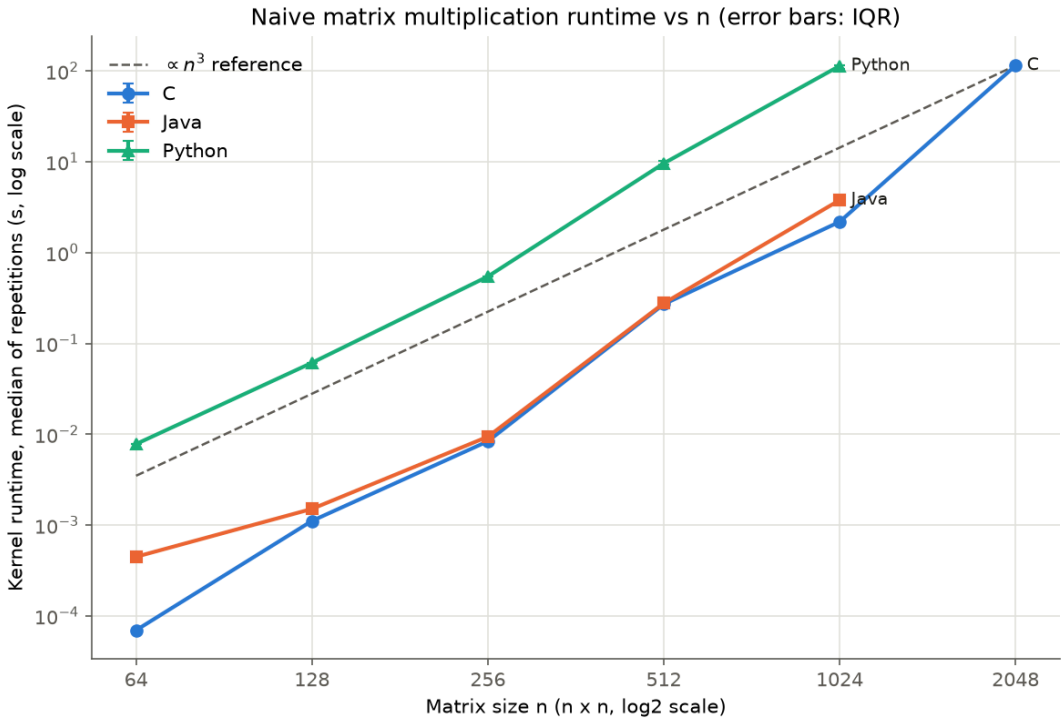


Figure 1. Kernel runtime against n (log-log scale), with error bars showing the IQR and a dashed n^3 reference line.

The largest feasible tested size was n = 2048 for C and n = 1024 for Java and Python. The largest size attempted was 4096 for C and 2048 for Java and Python.

Table 2. Growth ratio T(n)/T(n/2) of the median runtime. A purely cubic algorithm gives 8.

n	C	Java	Python
128	16.0	3.4	7.8
256	7.5	6.2	8.9
512	32.3	29.4	17.6
1024	8.1	13.7	11.8
2048	52.4	—	—

Table 3. Time per inner-loop iteration, T/n^3 (nanoseconds). A purely cubic cost gives a constant value. The last column gives the size of B ($8n^2$ bytes), the matrix read column by column.

n	C	Java	Python	Size of B
64	0.27	1.71	29.8	32 KiB
128	0.53	0.72	29.2	128 KiB
256	0.50	0.56	32.5	512 KiB
512	2.01	2.06	71.6	2 MiB
1024	2.03	3.52	105.8	8 MiB
2048	13.33	—	—	32 MiB

Variability was low: the IQR was below 2.5 % of the median for every C configuration up to n = 1024, and at most 6.4 % for Java. It increased for long runs: 4.8 % for C at n = 2048 and about 10 % for Python at n = 512 and 1024.

4.2 Memory

Table 4. Peak working set of the whole process (MiB).

n	C	Java	Python
64	3.9	46.8	13.3
128	4.2	48.4	14.2
256	5.3	53.4	20.5
512	9.7	71.8	42.8
1024	27.7	142.9	133.6
2048	99.8	—	—

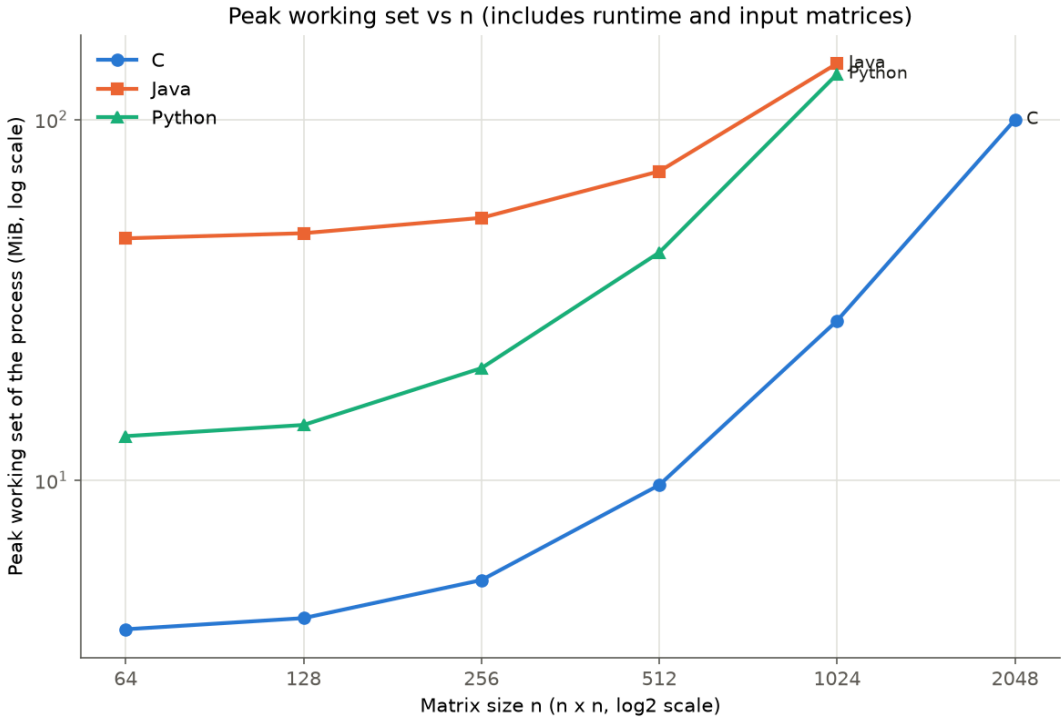


Figure 2. Peak working set against n.

4.3 Supplementary experiment: Java at n = 2048 with one warm-up

While analysing the main run, we found a bias in the budget design. Because the 900 s limit covered the whole process, Java (5 warm-ups + 5 repetitions = 10 multiplications) had a stricter effective limit than C and Python (1 + 5 = 6 multiplications). The Java timeout at n = 2048 could therefore reflect the number of warm-ups rather than the cost of one multiplication.

To test this, Java at n = 2048 was run again separately, with the same inputs, flags and pinning, one warm-up run (the same number as C) and a 2700 s budget. This run took place later on the same day as the main run (2026-10-14, 21:45), so its thermal and background conditions may differ. Its results are reported separately and are not merged into Tables 1–4.

Table 5. Supplementary run compared with C at n = 2048 from the main run.

	C (main run)	Java (supplementary)
Warm-ups	1	1
Median (s)	114.5	129.2
IQR (s)	5.5	9.8
Repetitions (s)	121.4, 112.3, 108.6, 114.5, 117.9	119.8, 124.7, 138.7, 134.4, 129.2
Peak working set (MiB)	99.8	303.3
Checksum	−23415.104678236432	−23415.104678236432

The whole Java process took about 13 minutes, so with one warm-up it would have fitted the original 900 s budget. With five warm-ups (about 10 × 129 s ≈ 21 min) it would not. This confirms that the main-run timeout was caused by the budget design.

5. Discussion

Does the runtime grow as n^3? (H1). Only partly. Each time n doubles, the work grows by 8, but the runtime did not always grow by 8. In C, it grew by about 8 when the matrix B stayed in the same cache level (7.5 from 128 to 256, and 8.1 from 512 to 1024). It grew much more when B stopped fitting in a cache level and moved to a slower one (16 from 64 to 128, 32.3 from 256 to 512, and 52.4 from 1024 to 2048).

Table 3 shows this clearly. In C, the time per iteration stays almost constant inside each level and jumps when B moves to the next one: about 0.27 ns while B fits in L1 (32 KiB < 48 KiB), about 0.5 ns in L2 (128–512 KiB < 1.25 MiB), about 2 ns in L3 (2–8 MiB < 24 MiB), and 13.3 ns once B no longer fits in L3 (32 MiB). In other words, the number of operations grows as n^3, but the cost of each operation depends on where the data is stored. This explanation matches the data, but we did not measure cache misses directly, so it is not proven. Part of the slowdown could also come from using only powers of two, which can make many elements of a column compete for the same cache positions (cache conflict misses); testing sizes such as n = 1000 would help to separate both effects.

Are C and Java faster than Python? (H2). Yes. Python was between 36 and 112 times slower than C (Table 1). Each Python iteration costs about 30 ns even when the data fits in cache, because the interpreter has to check types and create a new number object for every operation, while C executes a few machine instructions.

Java was almost as fast as C at n = 256 and 512 (1.12 and 1.02 times the C runtime), which makes sense because, after warm-up, the JIT compiler turns the loop into machine code like C. Java was slower at n = 64, possibly because fixed costs dominate at that size, or because the JIT had not fully optimised such a short loop. It was also slower at n = 1024 (1.73 times C), for reasons we did not investigate. At n = 2048 the gap became small again (1.13 times C, from the supplementary run): at that size both programs probably spend most of their time waiting for main memory, so the language matters less.

Is Python less affected by memory? (H3). Only partly. As we predicted, Python grew by about 8 for small sizes (7.8 and 8.9), but from 256 to 512 it grew by 17.6, and its time per iteration rose from about 30 ns to 106 ns. So memory also matters in Python, although less than in C. A likely reason is that Python stores each element as a pointer to a separate float object (about 32 bytes in total instead of 8), scattered in memory, so its matrices stop fitting in the caches at smaller sizes.

Memory. The memory measured for each process has two parts: a fixed part used by the language itself and the part used by the matrices. C uses almost nothing extra (about 4 MiB), and at n = 2048 its 99.8 MiB is close to the 96 MiB needed by the three matrices. The Java virtual machine uses about 47 MiB before storing any data, and Python about 13 MiB, but Python then grows faster than C because each element takes more memory. At n = 2048, Java used about three times the size of the matrices (303 MiB), possibly because of temporary buffers when loading the files; we did not measure the cause.

Variability. Short runs were very stable, while long runs varied more (up to about 10 % for Python at n = 1024). In the supplementary run, the Java times increased from one repetition to the next (from 120 s to 139 s). A lack of warm-up would make times decrease, not increase, so a more likely cause is that the laptop slowed down as it heated up (thermal throttling). We did not record the temperature or the processor frequency, so this remains a hypothesis.

Limitations.

- All results come from a single laptop, so they should not be read as “language X is always faster than language Y”.
- The cache explanation is based on timings; cache misses were not measured directly.
- Only powers of two were tested, which may amplify cache conflict effects.
- The memory metric includes the language runtime and the input data, not only the multiplication.
- The time limit penalised Java at n = 2048 (Section 4.3); the supplementary run used a different limit and was run at a different time.
- The benchmark did not record whether CPU pinning was applied successfully, so this cannot be confirmed for the reported data. The harness was corrected afterwards.
- Processor temperature and frequency were not recorded.

6. Conclusions

Within the tested scope (one Intel Core i7-12650H laptop, Windows 11, the stated compilers and runtimes, float64 inputs, n from 64 to 2048), the study establishes that:

1. The runtime of the naive i-j-k multiplication follows the cubic model only while matrix B stays within one level of the memory hierarchy. Each time it moves to a slower level, the time per iteration rises by a factor of about 2 to 7, so the memory layout dominates the deviation from n^3.
2. CPython was 36–112 times slower than C for the same algorithm, mainly because of per-operation interpreter overhead. Its time per iteration also increased with n, so memory effects are not absent.
3. After JIT warm-up, Java performed close to C for mid-range sizes. It was 1.7 times slower at n = 1024 and 1.13 times slower at n = 2048, where both appear to be memory-bound.
4. The peak working set grows with n^2 on top of a runtime-specific baseline, which is largest for the JVM.
5. Timeout budgets that include warm-ups can bias comparisons between languages with different warm-up requirements.

The study does not establish the exact cause of the per-level slowdowns (B no longer fitting in a cache level, or cache conflicts caused by power-of-two sizes), the reason for the Java–C gap at n = 1024, or the source of the extra Java memory. Hardware performance counters, non-power-of-two sizes and frequency logging would address these questions.

7. References

[1] Universidad de Las Palmas de Gran Canaria, "Individual Assignments — Matrix Multiplication," BIG DATA course, 2026-2027.

[2] Intel Corporation, "Intel Core i7-12650H Processor (24M Cache, up to 4.70 GHz) — Specifications," Intel ARK. [Online]. Available: https://www.intel.com/content/www/us/en/products/sku/226066/intel-core-i712650h-processor-24m-cache-up-to-4-70-ghz/specifications.html

[3] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Cambridge, MA, USA: Morgan Kaufmann, 2017.

[4] U. Drepper, "What Every Programmer Should Know About Memory," Red Hat, Inc., 2007.

[5] Oracle, "Java HotSpot Virtual Machine Performance Enhancements," in *Java Virtual Machine Guide*, Release 21. [Online]. Available: https://docs.oracle.com/en/java/javase/21/vm/java-hotspot-virtual-machine-performance-enhancements.html

[6] Python Software Foundation, "time — Time access and conversions," Python 3.13 documentation. [Online]. Available: https://docs.python.org/3.13/library/time.html

[7] G. Rodola, "psutil documentation." [Online]. Available: https://psutil.readthedocs.io

[8] Microsoft, "Acquiring high-resolution time stamps," Windows App Development documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/sysinfo/acquiring-high-resolution-time-stamps

[9] Microsoft, "GetSystemCpuSetInformation function (processthreadsapi.h)," Win32 API documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/processthreadsapi/nf-processthreadsapi-getsystemcpusetinformation

[10] A. Georges, D. Buytaert, and L. Eeckhout, "Statistically rigorous Java performance evaluation," in *Proc. 22nd ACM SIGPLAN Conf. Object-Oriented Programming Systems, Languages and Applications (OOPSLA)*, 2007, pp. 57–76.

[11] K. Goto and R. A. van de Geijn, "Anatomy of high-performance matrix multiplication," *ACM Trans. Math. Softw.*, vol. 34, no. 3, Art. 12, 2008.

[12] Microsoft, "GetLogicalProcessorInformationEx function (sysinfoapi.h)," Win32 API documentation. [Online]. Available: https://learn.microsoft.com/en-us/windows/win32/api/sysinfoapi/nf-sysinfoapi-getlogicalprocessorinformationex. Values recorded in `tools/cache_info.json`.

8. Reproducibility materials

All materials are in the repository above, in `assignment-1/`, at tag `assignment-1`:

- Source code: `c/`, `java/`, `python/`; benchmark harness `benchmark.py`; tests `run_tests.py`; NumPy cross-check `cross_check.py`.
- Input generation: `data/generate.py` (seeds and parameters in `data/README.md`).
- Configuration: `bench_config.json` (main run) and `bench_config_supplementary.json` (Section 4.3).
- Raw data: `results/raw_measurements.csv` and `results/supplementary/raw_measurements.csv`; environment metadata in `environment.json`.
- Analysis: `results/analyze.py` produces the summary tables and figures.
- Exact setup, build, test, run and analysis commands: `assignment-1/README.md`.

Generative AI disclosure

In this project, I used generative AI: Claude (Anthropic), through Claude Code. During the development process, I guided the AI in the experimental design, which I subsequently reviewed and approved. I also authorized it to implement the benchmark code (the three implementations, tests, harness and analysis scripts) based on that design. The AI assisted me in understanding both the code and the results, and helped draft the text of this report based on the measurements and analysis discussed. Finally, I conducted performance tests, identified bias stemming from the warm-up phase or resource

limitations, decided to carry out a supplementary experiment, edited the report, and verified the measurements and references.

I have a full understanding of both the code and the conclusions. All numerical results are derived from measurements stored in the repository; none were generated by the AI tool.